RaktaRun Deployment and Operations Guide

GitHub, Vercel, Render, Firebase and Supabase

Purpose and deployment model

This guide organizes the supplied deployment notes into a practical reference for source control, frontend and backend configuration, data storage, and release checks. The frontend is a React/Vite application; the API is an Express service; Firebase handles identity; and Supabase stores application records and certificate files.

Which service does what

SERVICE	ROLE IN THE DEPLOYMENT
GitHub	Source of truth for the application code. Changes pushed to the main branch trigger the configured deployments.
Vercel	Builds and serves the React/Vite frontend to users.
Render	Runs the Express API and receives frontend API requests.
Firebase	Provides browser-side sign-in and server-side identity verification through the Admin SDK.
Supabase	Stores PostgreSQL application records and private donor certificate uploads.

Request and release path

STAGE	WHAT HAPPENS
1. Change	A developer updates code and pushes a commit to GitHub main.
2. Deploy	Vercel rebuilds the frontend. Render installs dependencies, builds the API, and restarts the service.
3. Use	The browser loads the frontend from Vercel. API calls go to the Render URL configured in VITE_API_URL.
4. Persist	The API verifies identity and reads or writes records and certificate files through Firebase and Supabase.

Configuration boundary: VITE_ variables are exposed in the browser bundle. Firebase Admin credentials, Supabase secret keys, JWT_SECRET, OTP_SECRET, and provider API keys belong only in the backend environment.

1. Source control and deployment flow

Repository synchronization

The supplied notes identify the GitHub repository as **lifepulsecorp-art/RaktaRunRepo** and use **main** as the deployment branch. Keep reviewed application changes on this branch so the connected deployment services receive the same source revision.

Render CI/CD

Render is linked directly to main. A push starts dependency installation with `npm install --include=dev`, runs the configured build, and restarts the container with zero downtime. Backend configuration values are entered in Render's environment settings.

Vercel CI/CD

Vercel tracks the frontend source, compiles the React/Vite assets, and updates its edge network after a deployment. Frontend environment variables are build-time inputs, so changing one requires a new frontend build.

CORS synchronization

The production Vercel origin must be allowed by the API. The supplied workflow updated `server/src/index.ts` after the production URL was generated, then pushed the change so Render could rebuild with the origin whitelisted.

A practical change sequence

STEP	ACTION
1. Edit	Make the application change locally and inspect the files that will be committed.
2. Configure	Add or update provider environment variables in the correct service dashboard; do not commit secret values.
3. Commit	Commit the intended code and configuration-template changes to GitHub.
4. Observe	Check the Vercel frontend deployment and Render API deployment for build or startup errors.
5. Confirm	Open the production frontend and exercise a representative API-backed workflow.

Origin changes: If the production frontend domain changes, update the API CORS allowlist, deploy the API change, and then verify browser requests from the new origin.

2. Frontend environment variables | Vercel

Vite reads `VITE_`-prefixed values at build time and embeds them in browser code. Set these in the Vercel frontend environment for the intended deployment target, then redeploy after changes.

VARIABLE	PURPOSE AND CHECK
VITE_API_URL	Backend base URL. Supplied value: https://raktarunrepo.onrender.com. Confirm it points to the active Render service.
VITE_GOOGLE_MAPS_API_KEY	Browser key for the Google Maps JavaScript SDK used for maps, donor and hospital locations, radius circles, and routing.
VITE_FIREBASE_API_KEY	Firebase Client SDK key used by the browser for authentication.
VITE_FIREBASE_AUTH_DOMAIN	Firebase Authentication handler domain, typically `<project-id>.firebaseapp.com`.
VITE_FIREBASE_PROJECT_ID	Project identifier for the Firebase application.
VITE_FIREBASE_STORAGE_BUCKET	Default Google Cloud Storage bucket associated with the Firebase project.
VITE_FIREBASE_MESSAGING_SENDER_ID	Numeric project identifier used by Firebase Cloud Messaging for web push.
VITE_FIREBASE_APP_ID	Identifier for the web app registration inside the Firebase project.

Configuration checklist

- Set the backend URL to the production Render API, including the correct scheme and hostname.
- Match Firebase web values to one Firebase project; mixing values from different projects can break sign-in.
- Restrict browser API keys in their provider consoles where supported, including allowed sites and APIs.
- Add the production frontend domain to Firebase Authentication's authorized domains.
- Use Preview and Production environment scopes intentionally; preview builds should not accidentally use production-only credentials or data.

Public by design: A VITE_ value is visible to users in the built frontend. Never put a Firebase Admin private key, Supabase secret key, JWT secret, OTP secret, or mail-provider secret in a VITE_ variable.

3. Backend environment variables | Render

Keep these values in the Render API service environment. The Render dashboard should hold the real credentials; the repository should contain variable names and safe examples only.

GROUP	VARIABLE	PURPOSE
Email and notifications	BREVO_API_KEY	Authenticates transactional email requests for verification, OTP, and donor match alerts.
Email and notifications	BREVO_SENDER_EMAIL	Verified sender address shown in outgoing messages.
Email and notifications	BREVO_SENDER_NAME	Display name shown in outgoing email, such as RaktaRun or LifePulse.
Scheduling	ESCALATION_CRON_SECRET	Bearer token protecting /api/cron/escalation from untrusted callers.
Firebase Admin	FIREBASE_CLIENT_EMAIL	Service account identity used by server-side Firebase operations.
Firebase Admin	FIREBASE_PRIVATE_KEY	Service account RSA private key. Store as a Render secret; preserve escaped newline handling (\n).
Firebase Admin	FIREBASE_PROJECT_ID	Firebase project identifier for the Admin SDK.
Certificate review	GROQ_API_KEY	Authenticates Groq inference requests used for vision-assisted certificate review.
Session security	JWT_SECRET	Signs and verifies internal session tokens for staff, donors, and administrators.
Runtime	NODE_ENV	Set to production on Render for production runtime behavior.
Verification	OTP_SECRET	Secret used for hashing or generating one-time passwords.
Supabase	SUPABASE_URL	Project endpoint: https://ngoaaoopeulyoivjzlwby.supabase.co.
Supabase	SUPABASE_SECRET_KEY	Server-only secret. The supplied notes describe it as the service-role credential that bypasses RLS.
Supabase Storage	SUPABASE_CERTIFICATES_BUCKET	Bucket for donor certificate files; defaults to donor-certificates.

Before starting the API

- Set separate, long random values for JWT_SECRET and OTP_SECRET.
- Confirm Firebase Admin fields all refer to the same project as the frontend Firebase configuration.
- Confirm Supabase URL, secret key, and bucket name match the intended Supabase project.
- Configure the verified Brevo sender before relying on production email delivery.
- Add ESCALATION_CRON_SECRET before configuring an external scheduler.

Credential handling: Do not commit a service-account JSON file or paste private credentials into client configuration. If a real credential is exposed, revoke or rotate it in its provider console.

4. Supabase database and file storage

Application records use a JSONB payload so the current server data shapes can be stored while each row keeps a stable key and update timestamp.

Record structure

COLUMN	TYPE	PURPOSE
record_key	TEXT PRIMARY KEY	Stable key used by the application for lookup and upsert.
payload	JSONB NOT NULL	Existing application record stored as a JSON document.
updated_at	TIMESTAMPTZ	Timestamp updated when the record is written.

The 11 application tables

DOMAIN	TABLE
Accounts	users
Organizations	hospitals
Donor records	donors
Requests and matching	blood_requests, matches
Audit and buddy features	audit_logs, buddy_links, buddy_referrals
Verification and configuration	email_verifications, blocked_emails, system_config

Access control and RLS

The supplied notes describe schema and table grants for `anon`, `authenticated`, and `service_role` to resolve database permission errors. Row Level Security is enabled on the application tables. The server uses the Supabase secret credential for its application access path; keep that credential private because the notes identify it as a service-role credential that bypasses RLS.

Certificate storage

Uploaded donor certificate images and documents are stored in the `donor-certificates` bucket. The supplied design keeps the bucket private and serves certificate access through the authenticated admin API rather than making the bucket public.

Recommended first-time data setup

1. Create or select the intended Supabase project.
2. Run the checked-in schema migration from the Supabase SQL editor.
3. Configure the server-only Supabase URL, secret key, and bucket name.
4. Back up local JSON records before importing them.
5. Run the migration/import process once against the intended project and confirm expected records and certificate files.

Data safety: Treat donor, hospital, user, audit, verification, and certificate records as sensitive. Use a separate test project for previews and test data.

5. Release checklist and troubleshooting

Pre-deployment checklist

CHECK	READY WHEN
Repository	The intended source revision is on the deployment branch and no secrets or private data files are included.
Vercel frontend	Production VITE_ variables are present and the frontend rebuild has completed.
Render API	Backend secrets are configured, the build succeeds, and the service starts without credential errors.
Firebase	Client values and Admin SDK credentials point to the same project; production domain is authorized.
Supabase	Schema exists, server credentials are valid, and the certificate bucket is private.
CORS	The active Vercel production origin is allowed by the deployed API.
Data	Required records were migrated or seeded into the correct Supabase project.

Post-deployment smoke checks

- Open the production frontend and confirm its main page and static assets load.
- Sign in using the configured Firebase provider.
- Submit one representative API-backed action and confirm the API response succeeds.
- Verify a record is written to the intended Supabase table.
- If certificate upload is enabled, upload a safe test file and verify it remains in private storage.
- Review Vercel and Render deployment logs for build, environment, or runtime errors.

Common symptoms

SYMPTOM	FIRST CHECK
Frontend calls fail	Confirm VITE_API_URL and the Render service URL, then check browser CORS errors.
Firebase sign-in fails	Check matching project IDs, web config, enabled sign-in provider, authorized domain, and Admin credentials.
Database permission error	Check Supabase project URL/key, migration tables, RLS settings, and that the API uses the server-only secret.
Certificate access fails	Check bucket name, bucket privacy, upload limits, and API authorization.
Escalation does not run	Confirm ESCALATION_CRON_SECRET and that the scheduler sends the matching Bearer token to /api/cron/escalation.

The escalation endpoint is intended to be called by a trusted scheduler. Keep the bearer secret private and monitor scheduler responses and API logs.